

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Konstantin Kapinchev
University of Greenwich

Application Development

Lecture 5: Object-Oriented Techniques in Practice

In this lecture:

- **Overview**
- **Enumeration Type**
- **Generics**
- **Array of Objects**
- **List of Objects**
- **Date and Time**
- **Conclusions**

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Overview

- **The C# programming language provides a wide range of programming techniques, which extend the key OOP principles¹ and increase the computational capabilities of the language**
- **This lecture is focused on some of the more popular techniques, which are widely used in practice and improve the security, usability and maintenance of the code base of software systems**

¹Encapsulation, Abstraction, Inheritance and Polymorphism

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Enumeration Type

- **This is a data type which allows only a limited predefined values**
- **These values are called elements or labels**
- **The data type corresponds to the Categorical Variables in Statistics**
- **Most programming languages, including C#, support this data type**

Enumeration Type *continued*

- In C#:
 - Enumeration type is defined with the keyword "enum", followed by the name of the type which will be used in the program, followed by a list of the allowed values organised in a block
 - Enumeration type needs to be declared at namespace or class level, it cannot be declared within the scope of a method
 - The example on the next slide illustrates the use of enumeration types in C#

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Enumeration Type *continued*

- Example:

```
public enum Seasons
{
    Spring,
    Summer,
    Autumn,
    Winter
}
```

Name of the enumeration type, which will be used at a later stage in the program as a data type

```
// Declare variable of enum type
Seasons MySeason;
// Initialise the variable and display its value
MySeason = Seasons.Spring;
Console.WriteLine(MySeason);

// Initialise via integer and type cast
MySeason = (Seasons)1;
Console.WriteLine(MySeason);

// Increment the value of the enum type
// This might result in unexpected results if not used accurately
MySeason = MySeason + 1;
Console.WriteLine(MySeason);
```

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Generics

- **The concept of Generics in C# allows the developers to implement functionality, which works with multiple data types**
- **It improves code re-use and type safety**
- **The concept is based on the generic type parameter, which acts as a placeholder for specific data types, for example integer or string**
- **In this case, the generic type parameter allows the actual data type to be specified at a later stage**

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Generics *continued*

- **Generic Methods**
 - A generic method is declared with generic type parameters and it can return a generic type
 - When the method is called, the generic type is replaced with specific data type
 - The examples on the following slides illustrate the use of generics

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Generics *continued*

- Generic Methods Example 1

```
// Generic method swapping two variables  
// The actual type will be determined when the method is called
```

```
static void Swap<T>(ref T Left, ref T Right)  
{  
    T Temp;  
    Temp = Left;  
    Left = Right;  
    Right = Temp;  
}
```

This generic data type will be replaced with an actual data type when the method "Swap" is called, as demonstrated on the next slide

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Generics *continued*

- Generic Methods Example 1

```
// Test generic method with integers
int L = 5;
int R = 10;
Console.WriteLine("1. Left: {0}, Right: {1}", L, R);
Swap<int>(ref L, ref R); ←
Console.WriteLine("2. Left: {0}, Right: {1}", L, R);

// Test generic method with strings
string First = "First";
string Second = "Second";
Console.WriteLine("1. Left: {0}, Right: {1}", First, Second);
Swap<string>(ref First, ref Second); ←
Console.WriteLine("2. Left: {0}, Right: {1}", First, Second);
```

The method "Swap" is called with the "int" data type specified and two int parameters, namely L and R

The same method is called with two string variables

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Generics *continued*

- Generic Methods Example 2

```
// Generic method applying additionom
static T Addition<T>(T Left, T Right)
{
    dynamic X = Left;
    dynamic Y = Right;
    dynamic Z = X + Y;
    return(Z);
}
```

← The method "Addition" adds the two parameters, "Left" and "Right". Their type will be determined when the method is called.

← The method returns generic data type. When called, the method will return a value of the same data type as the parameters.

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Generics *continued*

- **Generic Methods Example 2**

```
// Test generic method with integers
Console.WriteLine("{0} + {1} = {2}", L, R, Addition<int>(L, R));
// Test generic method with strings
Console.WriteLine(Addition<string>(First, Second));
```

The method "Addition" is called with two integer variables, "L" and "R". The method will return integer value.

The same method is called with two string variables

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Arrays of Objects

- In many cases, a single class can describe multiple objects
- Those objects share the same structure but contain different data
- An array of objects allows a single data structure to store and process large amount of complex data with their corresponding functionality
- The use of arrays of objects is illustrated on the next slides

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Arrays of Objects *continued*

- An array of objects can be implemented by the following steps:
 1. Define a class (base class and derived classes)
 2. Declare an array, the type of the array is the class
 3. Initialise the array
 4. Utilise the class methods and data
- Example on the next slides illustrates the implementation of array of objects

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Arrays of Objects *continued*

- Example:

1. Define a class

```
class Person
{
    private string FirstName = "";
    private string LastName = "";

    public string getFirstName()
    { return (FirstName); }
    public string getLastName()
    { return (LastName); }

    public void setFirstName(string FN)
    { FirstName = FN; }
    public void setLastName(string LN)
    { LastName = LN; }
}
```

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Arrays of Objects *continued*

- Example:

2. Declare array

3. Initialise the array

```
int i;  
// Declare an array  
Person[] People = new Person[10];  
  
// Initialise the array  
for (i = 0; i < 10; i++)  
{  
    People [i] = new Person();  
}  
People[0].setFirstName("John");  
People[0].setLastName("Smith");  
People[1].setFirstName("Ann");  
People[1].setLastName("Jones");
```

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Arrays of Objects *continued*

- Example:

4. Utilise class methods and data

```
// Display all elements
i = 1;
foreach (Person a in People)
{
    Console.WriteLine("{0} {1} {2}", i, a.getFirstName(), a.getLastName());
    i = i + 1;
}
```

Potential issue:

The array has fixed number of elements, in this case 10 elements. Only the first two elements are initialised with data. Displaying the entire array will visualise uninitialised data

Application Development

Lecture 5: Object-Oriented Techniques in Practice

List of Objects

- An alternative solution is the utilisation of `List<T>` instead of array
- List as a data structure has a dynamic size, there is no fixed number of elements
- Displaying the entire List will display only initialised data
- The example on the next slide demonstrates an implementation of List of Objects

Application Development

Lecture 5: Object-Oriented Techniques in Practice

List of Objects *continued*

- Example:
- In this case, it is assumed that the array of objects is already initialised

```
// Declare a list of objects
List<Person> MyList = new List<Person>();

// Add the objects to the list
MyList.Add(People[0]);
MyList.Add(People[1]);
MyList.Add(People[2]);

// Display all items from the list
i = 1;
foreach (Person b in MyList)
{
    Console.WriteLine("{0} {1} {2}", i, b.getFirstName(), b.getLastName());
    i = i + 1;
}
```

Initialising the list via the array of objects does not create a limitation in the list implementation. These data can be provided through wide range of means, including via graphical user interface (Windows Forms)

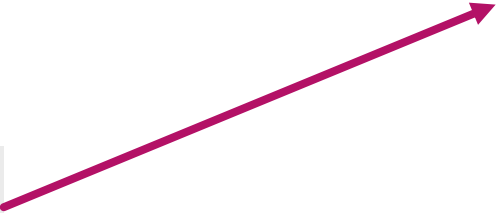
Application Development

Lecture 5: Object-Oriented Techniques in Practice

List of Objects *continued*

- Comparing the output

Constructed like this, the array displays a fixed number of elements.



```
1 John Smith
2 Ann Jones
3
4
5
6
7
8
9
10
```

Unlike the array, the list consists of only initialised elements



```
1 John Smith
2 Ann Jones
3 Mary Meyers
```

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Date and Time

- **Processing date and time accurately and consistently is of significant importance for a wide range of businesses and industries**
- **There are a number of challenges when working with date and time, including different time zones and daylight saving time**
- **The C# programming language provides support for working with date and time via the DateTime structure¹**

¹Note DateTime is a structure, not a class

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Date and Time *continued*

- Work with date and time starts with creating an object of the DateTime structure:

```
// Create an empty object  
DateTime a = new DateTime();  
Console.WriteLine(a.ToLongDateString());
```

- This generates the following result:
Monday, January 1, 0001

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Date and Time *continued*

- Alternatively, a date can be specified when an object is created

```
// Create an object  
DateTime a = new DateTime(2020, 1, 1);  
Console.WriteLine(a.ToLongDateString());
```

- This generates the following result:
Wednesday, January 1, 2020

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Date and Time *continued*

- Time can be added

```
// Create an object
DateTime a = new DateTime(2020, 1, 1, 16, 30, 0);
Console.WriteLine(a.ToLongDateString());
Console.WriteLine(a.ToLongTimeString());
```

- This generates the following result:
Wednesday, January 1, 2020
4:30:00 PM

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Date and Time *continued*

- Object can be initialised with the current date and time

```
// Create an object
DateTime a = new DateTime();
a = DateTime.Now;
Console.WriteLine(a.ToLongDateString());
Console.WriteLine(a.ToLongTimeString());
```

- This generates the following result:
Friday, February 17, 2023
10:47:54 AM

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Date and Time *continued*

- Some of the more popular operations with the `DateTime` structure include:
 - Comparisons:
 - `==`, `!=`, `>`, `>=`, `<`, `<=`
 - Convert date and time to string:
 - `ToLongDateString`, `ToLongTimeString`
 - `ToShortDateString`, `ToShortTimeString`
- And others ...

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Conclusions

- **This lecture discussed some popular programming techniques, which extend the functionality of the key principles of the object-oriented programming**
- **Those techniques have significant applications in practice**
- **Some of them can be found in other programming languages, such as C/C++ and Java, although different terminology might be used**

Application Development

Lecture 5: Object-Oriented Techniques in Practice

Conclusions

- **Further reading:**
 - **Enumeration Type:**
<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/enum>
 - **Generics**
<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/generics>
 - **Date and time**
<https://learn.microsoft.com/en-us/dotnet/api/system.datetime?view=net-7.0>